

Lab 01 - Cluster Reconnaissance with kubectl

Lab id	Lab 01
Day	1 - Cloud Native Foundations & Kubernetes Core Concepts
Duration	45 minutes
Namespace	kcna-lab01
Mapping	LO1 Develop a Kubernetes architectural proof of concept A1 Develop an architectural proof of concept K1 Process for developing proof of concepts

1. Objective

By the end of this lab you will be able to:

- Establish which cluster, which user and which namespace your kubectl is actually talking to, and change that scope deliberately.
- Enumerate a cluster's API surface with kubectl api-versions and kubectl api-resources, and tell namespaced from cluster-scoped kinds.
- Read the live OpenAPI schema with kubectl explain instead of guessing field names.
- Prove that kubectl is nothing but an HTTP client by exposing the raw REST request with --v=6.
- Retrieve the server-side truth of an object with kubectl get -o yaml and distinguish what **you** wrote from what the **control plane** added.
- Record a reconnaissance baseline and diff a live cluster against it - the first deliverable of any architectural proof of concept.

2. Prerequisites

- A running Kubernetes cluster, v1.30 or later. A single-node [kind](#) cluster is sufficient and is what this lab is written against.
- kubectl on your PATH, within one minor version of the server.
- No cluster-admin rights are required beyond create/read/delete inside your own namespace.

Confirm access before you start:

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kcna-qa-20260905-control-plane	Ready	control-plane	47m	v1.37.0

If kubectl version prints only a client version and then The connection to the server localhost:8080 was refused - did you specify the right host or port?, your kubeconfig is not being found. Stop here and fix it (see Troubleshooting).

Change into the lab directory - every relative path below is from there:

```
cd courseware/labs/lab-01-kubectl-and-cluster-recon
```

3. Scenario

Meridian Freight Pte Ltd is a Singapore-based regional logistics operator. Its Depot Portal - the web application that depot supervisors use to book inbound trailers - has been running on a pair of hand-managed VMs for six years. The platform team has been asked to produce an architectural proof of concept for moving Depot Portal onto Kubernetes.

You have just been handed a kubeconfig for a cluster nobody on your team built. Before a single line of Depot Portal YAML is written, the PoC has to answer one question honestly: what is actually on this cluster, and how do I find out?

That is this lab. You will build a reconnaissance record for the cluster and compare it against Meridian's standard baseline in data/core-api-inventory.tsv.

4. Step-by-step procedure

Step 1 - Find out who you are and where you are pointing

kubectl reads a kubeconfig file. That file contains **clusters** (an API server URL plus a CA), **users** (credentials) and **contexts** (a named pairing of the two, plus an optional default namespace).

List every context available to you:

```
kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
*	kind-kcna-qa	kind-kcna-qa	kind-kcna-qa	

The * marks the current context. Now print only the current context's details, with credentials redacted:

```
kubectl config view --minify
```

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://127.0.0.1:52913
    name: kind-kcna-qa
contexts:
- context:
    cluster: kind-kcna-qa
    user: kind-kcna-qa
    name: kind-kcna-qa
current-context: kind-kcna-qa
kind: Config
preferences: {}
users:
- name: kind-kcna-qa
  user:
    client-certificate-data: DATA+OMITTED
    client-key-data: DATA+OMITTED
```

Two things to notice:

- server: is an ordinary HTTPS URL. Everything kubectl does is an HTTP

request to that URL.

- There is no namespace: key in this context, so every namespaced command

you run defaults to the namespace default. That is the single most common source of "my Pod disappeared".

Print just the current context name (useful in scripts):

```
kubectl config current-context
```

```
kind-kcna-qa
```

Step 2 - Survey the nodes

```
kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
kcna-qa-20260905-control-plane	Ready	control-plane	47m	v1.37.0	172.18.0.2
EXTERNAL-IP OS-IMAGE KERNEL-VERSION CONTAINER-RUNTIME					
172.18.0.2 Debian GNU/Linux 12 (bookworm) 6.10.14-linuxkit containerd://2.1.4					

Read the last column. containerd://2.1.4 tells you the container runtime behind the CRI socket on this node.

Kubernetes never runs containers itself; it asks a CRI-conformant runtime to do it. You will come back to this in Lab 05.

Now look at what the node advertises to the scheduler:

```
kubectl get node -o jsonpath='{.items[0].status.allocatable}' | tr ',' '\n'
```

```
{
  "cpu": "10"
  "ephemeral-storage": "61202244Ki"
  "hugepages-1Gi": "0"
  "hugepages-2Mi": "0"
  "hugepages-32Mi": "0"
  "hugepages-64Ki": "0"
  "memory": "8039412Ki"
  "pods": "110"}

```

allocatable - not capacity - is the pool the scheduler may spend. Everything you will learn about requests and limits on Day 2 is arithmetic against these numbers.

Step 3 - Enumerate the API surface

A Kubernetes cluster is a set of REST APIs grouped into **API groups**. List the group/version pairs this server serves:

```
kubectl api-versions | sort | head -n 12
```

```
admissionregistration.k8s.io/v1
apiextensions.k8s.io/v1
apiregistration.k8s.io/v1
apps/v1
authentication.k8s.io/v1
authorization.k8s.io/v1
autoscaling/v1
autoscaling/v2
batch/v1
certificates.k8s.io/v1
coordination.k8s.io/v1
discovery.k8s.io/v1

```

Count them:

```
kubectl api-versions | wc -l
```

```
29
```

Now list the **kinds** those groups expose. The core group is the historical one and has an empty group name - that is why core objects use `apiVersion: v1` while Deployments use `apiVersion: apps/v1`:

```
kubectl api-resources --api-group=''
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
bindings		v1	true	Binding
configmaps	cm	v1	true	ConfigMap
endpoints	ep	v1	true	Endpoints
events	ev	v1	true	Event
limitranges	limits	v1	true	LimitRange
namespaces	ns	v1	false	Namespace
nodes	no	v1	false	Node
persistentvolumeclaims	pvc	v1	true	PersistentVolumeClaim
persistentvolumes	pvc	v1	false	PersistentVolume
Pods	po	v1	true	Pod
podtemplates		v1	true	PodTemplate
replicationcontrollers	rc	v1	true	ReplicationController
resourcequotas	quota	v1	true	ResourceQuota
secrets		v1	true	Secret
serviceaccounts	sa	v1	true	ServiceAccount
services	svc	v1	true	Service

The NAMESPACED column is the one that matters architecturally. Ask the API directly for everything that is not namespaced:

```
kubectl api-resources --namespaced=false --no-headers | awk '{print $1}' | head -n 12
```

```
componentstatuses
namespaces
nodes
persistentvolumes
mutatingwebhookconfigurations
validatingadmissionpolicies
validatingadmissionpolicybindings
validatingwebhookconfigurations
customresourcedefinitions
apiservices
selfsubjectreviews
tokenreviews
```

Cluster-scoped objects cannot be isolated per tenant with a namespace. That is a design constraint you must record in the PoC.

Finally, which verbs may you use on Pods?

```
kubectl api-resources --api-group='' -o wide | grep '^pods '
```

```
pods    po      v1      true    Pod      [create delete deletecollection get list patch update watch]
all
```

Step 4 - Read the schema instead of guessing

`kubectl explain` does not read a bundled cheat sheet. It downloads the server's OpenAPI v3 schema and walks it. That means it is always correct for *this* cluster's version.

```
kubectl explain pod
```

```

GROUP:
KIND:      Pod
VERSION:   v1

DESCRIPTION:
  Pod is a collection of containers that can run on a host. This resource is
  created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
  kind          <string>
  metadata      <ObjectMeta>
  spec          <PodSpec>
  status        <PodStatus>

```

Every Kubernetes object has these five top-level fields. Drill into spec:

```
kubectl explain pod.spec --recursive | head -n 20
```

```

GROUP:
KIND:      Pod
VERSION:   v1

FIELD: spec <PodSpec>

DESCRIPTION:
  Specification of the desired behavior of the pod.
FIELDS:
  activeDeadlineSeconds <integer>
  affinity              <Affinity>
    nodeAffinity        <NodeAffinity>
      preferredDuringSchedulingIgnoredDuringExecution <[]PreferredSchedulingTerm>
      preference      <NodeSelectorTerm>
        matchExpressions <[]NodeSelectorRequirement>
          key <string>
          operator <string>
          values <[]string>
        matchFields <[]NodeSelectorRequirement>
          key <string>
          operator <string>
          values <[]string>
      weight <integer>

```

Now the exact field you will need in Lab 02:

```
kubectl explain pod.spec.containers.resources
```

```

GROUP:
KIND:      Pod
VERSION:   v1

FIELD: resources <ResourceRequirements>

DESCRIPTION:
  Compute Resources required by this container. Cannot be updated.

FIELDS:
  claims      <[]ResourceClaim>
  limits      <map[string]Quantity>
  requests    <map[string]Quantity>

```

Try a field that does not exist - this is how you learn to trust the tool:

```
kubectl explain pod.spec.container
```

```
error: field "container" does not exist
```

Singular vs plural matters. containers is a list.

Step 5 - Create the lab namespace and its guardrails

Read the manifest first:

```
cat manifests/00-namespace.yaml
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: kcna-lab01
  labels:
    app.kubernetes.io/part-of: depot-portal
    kcna.tertiaryinfotech.com/lab: lab-01
    kcna.tertiaryinfotech.com/day: "1"
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/enforce-version: latest
```

Apply it:

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab01 created
```

Apply the namespaced policy objects. These give you something non-trivial to inspect, and they cap what this lab can consume:

```
kubectl apply -f manifests/10-namespace-guardrails.yaml
```

```
resourcequota/depot-recon-quota created
limitrange/depot-recon-defaults created
```

```
kubectl -n kcna-lab01 get resourcequota,limitrange
```

NAME	AGE	REQUEST
resourcequota/depot-recon-quota	8s	count/configmaps: 0/10, pods: 0/10, requests.cpu: 0/1, requests.memory: 0/1Gi
limitrange/depot-recon-defaults		limits.cpu: 0/2, limits.memory: 0/2Gi

NAME	CREATED AT
limitrange/depot-recon-defaults	2026-09-05T07:44:11Z

Step 6 - Turn the dataset into a ConfigMap

data/recon-checklist.csv is Meridian's standard recon checklist. Look at it:

```
head -n 5 data/recon-checklist.csv
```

```
# Meridian Freight Pte Ltd - Depot Portal cluster reconnaissance checklist
# Synthetic dataset for KCNA Lab 01. Consumed as ConfigMap `recon-checklist`.
check_id,area,question,kubectl_probe,evidence_expected
RC-001,identity,Which cluster and user is my context bound to?,kubectl config view --minify,
current-context and user name
RC-002,version,Do client and server minor versions match?,kubectl version,clientVersion and
serverVersion blocks
```

Before creating it for real, use `--dry-run=client -o yaml` to see exactly what `kubectl` would POST. This flag is the single most useful thing in the tool:

```
kubectl -n kcna-lab01 create configmap recon-checklist \
  --from-file=data/recon-checklist.csv \
  --dry-run=client -o yaml | head -n 12
```

```
apiVersion: v1
data:
  recon-checklist.csv: |
    # Meridian Freight Pte Ltd - Depot Portal cluster reconnaissance checklist
    # Synthetic dataset for KCNA Lab 01. Consumed as ConfigMap `recon-checklist`.
    check_id,area,question,kubectl_probe,evidence_expected
    RC-001,identity,Which cluster and user is my context bound to?,kubectl config view --
minify,current-context and user name
    RC-002,version,Do client and server minor versions match?,kubectl version,clientVersion
and serverVersion blocks
    RC-003,capacity,How many nodes are Ready and what is their role?,kubectl get nodes -o wide,
STATUS Ready and ROLES column
    RC-004,api,Which API groups and versions does the server serve?,kubectl api-versions,list
of group/version strings
    RC-005,api,Which resource kinds exist and are they namespaced?,kubectl api-resources,
NAMESPACED and KIND columns
    RC-006,schema,What fields may a Pod spec legally contain?,kubectl explain pod.spec,field
list with types
```

`--from-file=<path>` produces one key named after the file's basename. Now create it for real:

```
kubectl -n kcna-lab01 create configmap recon-checklist \
  --from-file=data/recon-checklist.csv
```

```
configmap/recon-checklist created
```

```
kubectl -n kcna-lab01 get configmap recon-checklist \
  -o jsonpath='{.data.recon-checklist\.csv}' | wc -l
```

```
15
```

The `\.` escape is required: `jsonpath` treats `.` as a path separator, so a key containing a dot must be escaped.

Step 7 - Deploy the reconnaissance Pod

Do not run this yet if you want to see the failure injection first - skip to section 7, then come back. If you are running straight through:

```
kubectl apply -f manifests/20-recon-shell.yaml
```

```
pod/recon-shell created
```

```
kubectl -n kcna-lab01 wait --for=condition=Ready pod/recon-shell --timeout=90s
```

```
pod/recon-shell condition met
```

```
kubectl -n kcna-lab01 get pod recon-shell
```

NAME	READY	STATUS	RESTARTS	AGE
recon-shell	1/1	Running	0	14s

The container read the mounted CSV at start-up. Prove it:

```
kubectl -n kcna-lab01 logs recon-shell | tail -n 3
```

```
RC-012,events,What has the control plane recently done here?,kubectl get events -n kcna-lab01 -
-sort-by=.lastTimestamp,Scheduled Pulled Created Started
[recon-shell] checklist rows: 13
[recon-shell] holding open for kubectl exec
```

13 is the count of non-comment lines: the CSV header row plus the 12 RC-0nn records. The container computed it with `grep -vc '^#' against the mounted file, so this line is direct evidence that the ConfigMap really was mounted and read.`

And from inside the container:

```
kubectl -n kcna-lab01 exec recon-shell -- ls -l /etc/depot
```

```
total 0
lrwxrwxrwx 1 root root 26 Sep 5 07:45 recon-checklist.csv -> ../data/recon-
checklist.csv
```

That symlink is how the kubelet does atomic ConfigMap updates - the real file lives under a timestamped `..2026_09_05_07_45_10.123456789` directory and `..data` is flipped between them.

Step 8 - Read the server-side truth of an object

```
kubectl -n kcna-lab01 get pod recon-shell -o yaml | head -n 30
```

```

apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Pod","metadata":{"...}}
  creationTimestamp: "2026-09-05T07:45:09Z"
  labels:
    app.kubernetes.io/name: recon-shell
    app.kubernetes.io/part-of: depot-portal
    kcna.tertiaryinfotech.com/lab: lab-01
  name: recon-shell
  namespace: kcna-lab01
  resourceVersion: "4127"
  uid: 8f1cb0f2-6d9b-4b8f-9f0a-4b0c8a1f2d31
spec:
  containers:
    - args:
      - |
        echo "[recon-shell] Meridian Freight depot reconnaissance checklist"
      ...
    image: busybox:1.36
    imagePullPolicy: IfNotPresent
    name: shell

```

You wrote none of creationTimestamp, resourceVersion, uid, or the whole status: block at the bottom. Those are control-plane defaulted and controller-written fields. The discipline for a PoC is: your Git repository holds *desired state*; kubectl get -o yaml shows *desired + defaulted + observed*.

Isolate just the observed part:

```

kubectl -n kcna-lab01 get pod recon-shell \
-o jsonpath='{.status.phase}{"\t"}{.status.podIP}{"\t"}{.spec.nodeName}{"\n"}'

```

```
Running 10.244.0.8 kcna-qa-20260905-control-plane
```

Step 9 - Prove kubectl is just an HTTP client

Raise the log verbosity to 6. Level 6 prints one line per HTTP request:

```
kubectl -n kcna-lab01 get pods --v=6
```

```

I0905 15:44:02.118374 62311 loader.go:395] Config loaded from file: /root/.kube/config
I0905 15:44:02.146902 62311 round_tripper.go:553] GET https://127.0.0.1:52913/api/v1/
namespaces/kcna-lab01/pods?limit=500 200 OK in 21 milliseconds
NAME      READY   STATUS    RESTARTS   AGE
recon-shell 1/1     Running   0           64s

```

Read the URL carefully:

Segment	Meaning
/api/v1	the core group (no group name) at version v1
/namespaces/kcna-lab01	the namespace scope
/pods	the resource, plural, lower-case - exactly the NAME column of api-resources
?limit=500	server-side pagination

Now do the same for a resource in a named group:

```
kubectl -n kcna-lab01 get deployments --v=6 2>&1 | grep round-trippers
```

```
I0905 15:45:31.402118 62410 round-trippers.go:553] GET https://127.0.0.1:52913/apis/apps/v1/namespaces/kcna-lab01/deployments?limit=500 200 OK in 8 milliseconds
```

/apis/apps/v1/... - note /apis (plural) for named groups versus /api for the core group. This asymmetry is a historical artefact and it appears in the KCNA exam.

You can skip kubectl's object model entirely and hit the API through the built-in proxy:

```
kubectl get --raw /api/v1/namespaces/kcna-lab01/pods/recon-shell | head -c 300; echo
```

```
{"kind":"Pod","apiVersion":"v1","metadata":{"name":"recon-shell","namespace":"kcna-lab01","uid":"8f1cb0f2-6d9b-4b8f-9f0a-4b0c8a1f2d31","resourceVersion":"4127","creationTimestamp":"2026-09-05T07:45:09Z","labels":{"app.kubernetes.io/name":"recon-shell",
```

Step 10 - Diff the cluster against Meridian's baseline

```
kubectl api-resources --api-group='' --no-headers | awk '{print $1}' | sort > /tmp/live-core.txt
grep -v '^#' data/core-api-inventory.tsv | cut -f1 | sort > /tmp/baseline-core.txt
comm -23 /tmp/baseline-core.txt /tmp/live-core.txt
```

Empty output is the pass condition: nothing Meridian expects is missing. Look at the other direction:

```
comm -13 /tmp/baseline-core.txt /tmp/live-core.txt
```

```
bindings
endpoints
```

endpoints is served but excluded from the baseline on purpose: v1 Endpoints was deprecated in Kubernetes v1.33 in favour of discovery.k8s.io/v1 EndpointSlice. You will use EndpointSlice on Day 3.

Step 11 - Look at what the control plane has been doing

```
kubectl -n kcna-lab01 get events --sort-by=.lastTimestamp
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m	Normal	Scheduled	pod/recon-shell	Successfully assigned kcna-lab01/recon-shell to kcna-qa-20260905-control-plane
2m	Normal	Pulled	pod/recon-shell	Container image "busybox:1.36" already present on machine
2m	Normal	Created	pod/recon-shell	Created container: shell
2m	Normal	Started	pod/recon-shell	Started container shell

Events are ordinary API objects with a TTL (one hour by default). They are the first place to look when something did not happen.

5. Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script asserts, in order: API reachability; the namespace, ResourceQuota and LimitRange; the ConfigMap and its key and content; the Pod's phase, readiness and log evidence that the mounted CSV was read; the full baseline diff against data/core-api-inventory.tsv; that kubectl explain returns live schema; and that --v=6 prints the expected REST path.

Expected tail:

```
Result: 13 passed, 0 failed
```

The full annotated evidence is in [`verification/expected-output.md`](#).

6. Failure injection - the Pod that never starts

Do this deliberately. Delete the ConfigMap and recreate the Pod so the volume source is missing.

```
kubectl -n kcna-lab01 delete pod recon-shell --ignore-not-found
kubectl -n kcna-lab01 delete configmap recon-checklist --ignore-not-found
kubectl apply -f manifests/20-recon-shell.yaml
```

```
pod/recon-shell deleted
configmap "recon-checklist" deleted
pod/recon-shell created
```

Now watch:

```
kubectl -n kcna-lab01 get pod recon-shell -w
```

NAME	READY	STATUS	RESTARTS	AGE
recon-shell	0/1	ContainerCreating	0	5s
recon-shell	0/1	ContainerCreating	0	35s

Press Ctrl+C. The Pod is stuck. kubectl logs is useless here - there is no container yet:

```
kubectl -n kcna-lab01 logs recon-shell
```

```
Error from server (BadRequest): container "shell" in pod "recon-shell" is waiting to start:
ContainerCreating
```

The evidence lives in the events, so describe the Pod:

```
kubectl -n kcna-lab01 describe pod recon-shell | tail -n 12
```

```

Events:
  Type            Reason            Age           From              Message
  ----            -
  Normal          Scheduled         62s           default-scheduler Successfully assigned kcna-lab01/
recon-shell to kcna-qa-20260905-control-plane
  Warning         FailedMount       30s (x8 over 62s) kubelet           MountVolume.SetUp failed for
volume "checklist" : configmap "recon-checklist" not found
  Warning         FailedMount       17s           kubelet           Unable to attach or mount
volumes: unmounted volumes=[checklist], unattached volumes=[], failed to process volumes=[]:
timed out waiting for the condition

```

Diagnosis

Read the three facts in order:

1. Scheduled succeeded - so this is not a scheduling problem. The API server accepted the object and the scheduler placed it on a node. Nothing is wrong with resources, taints or selectors.
2. The failure is reported by kubelet, not by the scheduler or the API server. The node is trying and failing to build the container's filesystem.
3. MountVolume.SetUp failed ... configmap "recon-checklist" not found names the volume (checklist) and the missing object exactly.

The API server accepted a Pod that references a ConfigMap which does not exist. Kubernetes does not validate cross-object references at admission time. The object is syntactically valid; the reconciliation loop just cannot converge. It will keep retrying forever, and it will succeed the moment the ConfigMap appears - you do not need to recreate the Pod.

Fix it without touching the Pod:

```
kubectl -n kcna-lab01 create configmap recon-checklist \
  --from-file=data/recon-checklist.csv
```

```
configmap/recon-checklist created
```

```
kubectl -n kcna-lab01 wait --for=condition=Ready pod/recon-shell --timeout=120s
kubectl -n kcna-lab01 get pod recon-shell
```

```

pod/recon-shell condition met
NAME          READY   STATUS    RESTARTS   AGE
recon-shell   1/1     Running   0           2m14s

```

Note the AGE - it never restarted. The kubelet simply retried the mount. This is the level-triggered, eventually-consistent behaviour that defines Kubernetes controllers, and it is worth more to your PoC than any diagram.

7. Troubleshooting

Symptom	Likely cause	Fix	
The connection to the server localhost:8080 was refused	No kubeconfig found - \$KUBECONFIG unset and ~/.kube/config absent	export KUBECONFIG=/path/to/kubeconfig, then re-run kubectl version	
Error from server (NotFound): pods "recon-shell" not found	You omitted -n kcna-lab01; the request went to namespace default	Add -n kcna-lab01, or set a default with kubectl config set-context --current --namespace=kcna-lab01	
error: field "container" does not exist from kubectl explain	Singular/plural or path typo	Walk the path one level at a time: kubectl explain pod.spec first, then pick the exact field name from the FIELDS list	
Pod stuck ContainerCreating, logs returns BadRequest ... waiting to start	Volume source (ConfigMap/Secret/PVC) missing or misnamed	kubectl -n kcna-lab01 describe pod <name> and read the FailedMount event; create the missing object	
error: exec plugin: invalid apiVersion "client.authentication.k8s.io/v1alpha1"	kubeconfig written by a much older CLI than your kubectl	Regenerate the kubeconfig with the current tool, or upgrade/downgrade kubectl to within one minor of the server	
Error from server (Forbidden): pods is forbidden: User "..." cannot list resource "pods"	RBAC - your identity has no permission in that namespace	kubectl auth can-i list pods -n kcna-lab01; ask for a Role/RoleBinding (covered in Lab 14)	
kubectl api-resources prints error: unable to retrieve the complete list of server APIs	An APIService (usually a metrics or custom-API extension) is unavailable	`kubectl get apiservices \	grep -v True` to find the broken one; the rest of the output is still valid
ConfigMap created but the Pod sees an empty directory	--from-file pointed at a directory, or the key name does not match the file you cat	kubectl -n kcna-lab01 get cm recon-checklist -o yaml and check the keys under data:	

8. Cleanup

Delete only this lab's namespace. Everything created above is namespaced, so this removes all of it.

```
kubectl delete namespace kcna-lab01
```

```
namespace "kcna-lab01" deleted
```

```
kubectl get namespace kcna-lab01
```

```
Error from server (NotFound): namespaces "kcna-lab01" not found
```

Do not run kubectl delete without -n/--namespace scoping, and never with --all-namespaces.

9. What you learned

- A context binds a cluster, a user and (optionally) a default namespace.

kubectl config view --minify is the fastest way to answer "where am I?".

- The API is organised into groups; the historical *core* group has an empty

name and lives at /api/v1, while every named group lives at /apis/<group>/<version>.

- kubectl api-resources is the authoritative map of a cluster's kinds,

including the namespaced vs cluster-scoped distinction that governs multi-tenant design.

- kubectl explain reads the live OpenAPI schema, so it is never out of date

for the cluster in front of you.

- --v=6 and kubectl get --raw show that kubectl is a thin, well-behaved

HTTP client; anything it can do, a controller or a CI job can do.

- kubectl get -o yaml returns desired plus defaulted plus observed

state. Only the first of those belongs in Git.

- Missing cross-object references are not rejected at admission. They surface

as kubelet events and resolve themselves when the referenced object appears.

10. Further reading

- [Command line tool \(kubectl\)](#)
- [kubectl Quick Reference](#)
- [Configure Access to Multiple Clusters](#)
- [Kubernetes API Concepts](#)
- [API Groups and Versioning](#)
- [Namespaces](#)
- [Resource Quotas](#) and [Limit Ranges](#)
- [Kubernetes Object Management](#)
- [Endpoints deprecation \(v1.33\)](#)